Mauricio de Souza Coutinho

Exercício 1:

Telefones do paciente (violação de 1FN):

Problema: o atributo telefone guardava vários números na mesma célula (atributo multivalorado). Solução: os telefones foram para a tabela **telefone_paciente**, com chave composta (id_paciente, telefone).

```sql
DROP DATABASE IF EXISTS t10_ex01;
CREATE DATABASE t10_ex01;
USE t10_ex01;

CREATE TABLE paciente (
  id_paciente INT PRIMARY KEY,
  nome        VARCHAR(100) NOT NULL
);

CREATE TABLE telefone_paciente (
  id_paciente INT         NOT NULL,
  telefone    VARCHAR(20) NOT NULL,
  PRIMARY KEY (id_paciente, telefone),
  CONSTRAINT fk_tel_paciente FOREIGN KEY (id_paciente)
    REFERENCES paciente (id_paciente)
);

INSERT INTO paciente VALUES (1,'Ana Lima'),(2,'Bruno Reis'),(3,'Carla Luz');
INSERT INTO telefone_paciente VALUES
  (1,'1191111'),(1,'1192222'),
  (2,'1193333'),
  (3,'1194444'),(3,'1195555'),(3,'1196666');
```

Validação: reconstrói paciente + telefone (6 linhas, nenhum NULL):

```sql
-- Validação: reconstrói paciente + telefone (6 linhas, nenhum NULL)
SELECT p.id_paciente, p.nome, t.telefone
FROM paciente p
JOIN telefone_paciente t ON t.id_paciente = p.id_paciente
ORDER BY p.id_paciente, t.telefone;
```

Result Grid Filter Rows: Export: Wrap Cell Content: IA

id_paciente	nome	telefone
1	Ana Lima	1191111
1	Ana Lima	1192222
2	Bruno Reis	1193333
3	Carla Luz	1194444
3	Carla Luz	1195555
3	Carla Luz	1196666

Exercício 2:

Produtos como lista no pedido (violação de 1FN):

Problema: a lista de produtos e quantidades ficava em uma única coluna do pedido. Solução: cada produto vira uma linha em **item_pedido** (id_pedido, id_produto, quantidade).

```sql
DROP DATABASE IF EXISTS t10_ex02;
CREATE DATABASE t10_ex02;
USE t10_ex02;

CREATE TABLE cliente (
  id_cliente INT PRIMARY KEY
);

CREATE TABLE produto (
  id_produto INT PRIMARY KEY
);

CREATE TABLE pedido (
  id_pedido  INT  PRIMARY KEY,
  data       DATE NOT NULL,
  id_cliente INT  NOT NULL,
  CONSTRAINT fk_pedido_cliente FOREIGN KEY (id_cliente)
    REFERENCES cliente (id_cliente)
);

CREATE TABLE item_pedido (
  id_pedido  INT NOT NULL,
  id_produto INT NOT NULL,
  quantidade INT NOT NULL,
  PRIMARY KEY (id_pedido, id_produto),
  CONSTRAINT fk_item_pedido  FOREIGN KEY (id_pedido)  REFERENCES pedido  (id_pedido),
  CONSTRAINT fk_item_produto FOREIGN KEY (id_produto) REFERENCES produto (id_produto),
  CONSTRAINT ck_item_qtd CHECK (quantidade > 0)
);

INSERT INTO cliente VALUES (10),(11);
INSERT INTO produto VALUES (7),(8),(9);
INSERT INTO pedido VALUES
  (101,'2026-10-01',10),
  (102,'2026-10-02',11),
  (103,'2026-10-03',10);
INSERT INTO item_pedido VALUES
  (101,7,2),(101,9,1),
  (102,8,3),
  (103,7,1),(103,8,2),(103,9,1);
```

Validação: itens do pedido 103 (3 linhas: 7x1, 8x2, 9x1):

```sql
-- Validação: itens do pedido 103 (3 linhas: 7x1, 8x2, 9x1)
SELECT p.id_pedido, p.data, p.id_cliente, i.id_produto, i.quantidade
FROM pedido p
JOIN item_pedido i ON i.id_pedido = p.id_pedido
WHERE p.id_pedido = 103
ORDER BY i.id_produto;
```

Result Grid Filter Rows: Export: Wrap Cell Content: IA

id_pedido	data	id_cliente	id_produto	quantidade
103	2026-10-03	10	7	1
103	2026-10-03	10	8	2
103	2026-10-03	10	9	1

Exercício 3:

Matrícula com dependências parciais (2FN):

Chave: {id_aluno, id_turma}. nome_aluno depende só de id_aluno e semestre depende só de id_turma (dependências parciais). Só a nota depende da chave inteira, por isso fica em **matricula**.

```sql
DROP DATABASE IF EXISTS t10_ex03;
CREATE DATABASE t10_ex03;
USE t10_ex03;

CREATE TABLE aluno (
  id_aluno   INT PRIMARY KEY,
  nome_aluno VARCHAR(100) NOT NULL
);

CREATE TABLE turma (
  id_turma INT PRIMARY KEY,
  semestre CHAR(6) NOT NULL
);

CREATE TABLE matricula (
  id_aluno INT          NOT NULL,
  id_turma INT          NOT NULL,
  nota     DECIMAL(4,2) NULL,
  PRIMARY KEY (id_aluno, id_turma),
  CONSTRAINT fk_mat_aluno FOREIGN KEY (id_aluno) REFERENCES aluno (id_aluno),
  CONSTRAINT fk_mat_turma FOREIGN KEY (id_turma) REFERENCES turma (id_turma),
  CONSTRAINT ck_mat_nota  CHECK (nota BETWEEN 0 AND 10)
);

INSERT INTO aluno VALUES (1,'Ana Lima'),(2,'Bruno Reis');
INSERT INTO turma VALUES (501,'2026.2'),(502,'2026.2');
INSERT INTO matricula VALUES (1,501,8.5),(1,502,9.0),(2,501,7.0);
```

Validação: reconstrói a relação original (3 linhas):

```sql
-- Validação: reconstrói a relação original (3 linhas)
SELECT a.id_aluno, a.nome_aluno, t.id_turma, t.semestre, m.nota
FROM matricula m
JOIN aluno a ON a.id_aluno = m.id_aluno
JOIN turma t ON t.id_turma = m.id_turma
ORDER BY a.id_aluno, t.id_turma;
```

Result Grid Filter Rows: Export: Wrap Cell Content: IA

id_aluno	nome_aluno	id_turma	semestre	nota
1	Ana Lima	501	2026.2	8.50
1	Ana Lima	502	2026.2	9.00
2	Bruno Reis	501	2026.2	7.00

Exercício 4:

Itens de compra com cliente e produto repetidos (2FN/3FN):

Na chave {id_compra, id_produto}, nome_produto depende só de id_produto (parcial) e nome_cliente depende de id_compra através de id_cliente (transitiva). Separação em cliente, produto, compra e item_compra.

```sql
DROP DATABASE IF EXISTS t10_ex04;
CREATE DATABASE t10_ex04;
USE t10_ex04;

CREATE TABLE cliente (
  id_cliente   INT PRIMARY KEY,
  nome_cliente VARCHAR(100) NOT NULL
);

CREATE TABLE produto (
  id_produto   INT PRIMARY KEY,
  nome_produto VARCHAR(100) NOT NULL
);

CREATE TABLE compra (
  id_compra  INT PRIMARY KEY,
  id_cliente INT NOT NULL,
  CONSTRAINT fk_compra_cliente FOREIGN KEY (id_cliente) REFERENCES cliente (id_cliente)
);

CREATE TABLE item_compra (
  id_compra  INT NOT NULL,
  id_produto INT NOT NULL,
  qtd        INT NOT NULL,
  PRIMARY KEY (id_compra, id_produto),
  CONSTRAINT fk_ic_compra  FOREIGN KEY (id_compra)  REFERENCES compra  (id_compra),
  CONSTRAINT fk_ic_produto FOREIGN KEY (id_produto) REFERENCES produto (id_produto),
  CONSTRAINT ck_ic_qtd CHECK (qtd > 0)
);

INSERT INTO cliente VALUES (10,'Ana'),(11,'Bruno');
INSERT INTO produto VALUES (7,'Mouse'),(8,'Teclado');
INSERT INTO compra  VALUES (900,10),(901,11);
INSERT INTO item_compra VALUES (900,7,2),(900,8,1),(901,7,3);
```

Validação: quantidade total por compra (900 -> 3, 901 -> 3):

```sql
-- Validação: quantidade total por compra (900 -> 3, 901 -> 3)
SELECT c.id_compra, cl.nome_cliente, SUM(i.qtd) AS qtd_total
FROM compra c
JOIN cliente cl    ON cl.id_cliente = c.id_cliente
JOIN item_compra i ON i.id_compra   = c.id_compra
GROUP BY c.id_compra, cl.nome_cliente
ORDER BY c.id_compra;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

id_compra	nome_cliente	qtd_total
900	Ana	3
901	Bruno	3

Exercício 5:

Funcionário, departamento e gerente (3FN):

Dependência transitiva: id_func -> id_depto -> gerente. O gerente pertence ao departamento, não ao funcionário, então foi movido para **departamento** (FK para gerente).

```sql
DROP DATABASE IF EXISTS t10_ex05;
CREATE DATABASE t10_ex05;
USE t10_ex05;

CREATE TABLE gerente (
  id_gerente INT PRIMARY KEY,
  nome       VARCHAR(100) NOT NULL
);

CREATE TABLE departamento (
  id_depto   INT PRIMARY KEY,
  nome_depto VARCHAR(60) NOT NULL UNIQUE,
  id_gerente INT NOT NULL,
  CONSTRAINT fk_depto_gerente FOREIGN KEY (id_gerente) REFERENCES gerente (id_gerente)
);

CREATE TABLE funcionario (
  id_func  INT PRIMARY KEY,
  nome     VARCHAR(100) NOT NULL,
  id_depto INT NOT NULL,
  CONSTRAINT fk_func_depto FOREIGN KEY (id_depto) REFERENCES departamento (id_depto)
);

INSERT INTO gerente      VALUES (7,'Marcos'),(8,'Paula');
INSERT INTO departamento VALUES (20,'TI',7),(30,'RH',8);
INSERT INTO funcionario  VALUES (1,'Lia',20),(2,'Caio',20),(3,'Rui',30);
```

Validação: funcionários com departamento e gerente (3 linhas):

```sql
-- Validação: funcionários com departamento e gerente (3 linhas)
SELECT f.id_func, f.nome, d.id_depto, d.nome_depto AS depto,
       g.id_gerente, g.nome AS gerente
FROM funcionario f
JOIN departamento d ON d.id_depto   = f.id_depto
JOIN gerente g      ON g.id_gerente = d.id_gerente
ORDER BY f.id_func;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

id_func	nome	id_depto	depto	id_gerente	gerente
1	Lia	20	TI	7	Marcos
2	Caio	20	TI	7	Marcos
3	Rui	30	RH	8	Paula

Exercício 6:

Ordem de manutenção com cadastros misturados (3FN):

Nome do cliente, modelo do equipamento e nome do técnico dependiam dos seus próprios ids, não da OS (transitivas). Cada cadastro virou tabela própria e a OS guarda só as FKs e as horas, com CHECK para horas entre 0 e 24.

```sql
DROP DATABASE IF EXISTS t10_ex06;
CREATE DATABASE t10_ex06;
USE t10_ex06;

CREATE TABLE cliente (
  id_cliente   INT PRIMARY KEY,
  nome_cliente VARCHAR(100) NOT NULL
);

CREATE TABLE equipamento (
  id_equip INT PRIMARY KEY,
  modelo   VARCHAR(30) NOT NULL
);

CREATE TABLE tecnico (
  id_tecnico   INT PRIMARY KEY,
  nome_tecnico VARCHAR(100) NOT NULL
);

CREATE TABLE ordem_servico (
  id_os      INT PRIMARY KEY,
  id_cliente INT NOT NULL,
  id_equip   INT NOT NULL,
  id_tecnico INT NOT NULL,
  horas      DECIMAL(5,2) NOT NULL,
  CONSTRAINT fk_os_cliente FOREIGN KEY (id_cliente) REFERENCES cliente (id_cliente),
  CONSTRAINT fk_os_equip   FOREIGN KEY (id_equip)   REFERENCES equipamento (id_equip),
  CONSTRAINT fk_os_tecnico FOREIGN KEY (id_tecnico) REFERENCES tecnico (id_tecnico),
  CONSTRAINT ck_os_horas   CHECK (horas > 0 AND horas <= 24)
);

INSERT INTO cliente     VALUES (10,'Ana'),(11,'Rui');
INSERT INTO equipamento VALUES (80,'X100'),(81,'Z200'),(82,'X100');
INSERT INTO tecnico     VALUES (7,'Ivo'),(8,'Bia');
INSERT INTO ordem_servico VALUES
  (501,10,80,7,2.5),
  (502,10,81,8,1.0),
  (503,11,82,7,3.0);
```

Validação: horas trabalhadas por técnico (Ivo 5.5 | Bia 1.0):

```sql
-- Validação: horas trabalhadas por técnico (Ivo 5.5 | Bia 1.0)
SELECT t.id_tecnico, t.nome_tecnico, SUM(os.horas) AS total_horas
FROM tecnico t
JOIN ordem_servico os ON os.id_tecnico = t.id_tecnico
GROUP BY t.id_tecnico, t.nome_tecnico
ORDER BY total_horas DESC;
```

Result Grid Filter Rows: Export: Wrap Cell Content:

id_tecnico	nome_tecnico	total_horas
7	Ivo	5.50
8	Bia	1.00

Teste do CHECK (deve falhar com erro 3819):

```sql
-- Teste do CHECK (deve falhar com erro 3819):
INSERT INTO ordem_servico VALUES (504,11,82,8,-1.0);
```

Output Action Output

#	Time	Action	Message
1	21:14:07	INSERT INTO ordem_servico VALUES (504,11,82,8,-1.0)	Error Code: 3819. Check constraint 'ck_os_horas' is violated.

Exercício 7:

Venda, produto e categoria (2FN/3FN):

nome_produto e categoria dependiam só do produto (parcial na chave {venda, produto}) e nome_categoria dependia da categoria (transitiva). Separação em cliente, categoria, produto, venda e item_venda.

```sql
DROP DATABASE IF EXISTS t10_ex07;
CREATE DATABASE t10_ex07;
USE t10_ex07;

CREATE TABLE cliente (
  id_cliente   INT PRIMARY KEY,
  nome_cliente VARCHAR(100) NOT NULL
);

CREATE TABLE categoria (
  id_categoria   INT PRIMARY KEY,
  nome_categoria VARCHAR(60) NOT NULL UNIQUE
);

CREATE TABLE produto (
  id_produto   INT PRIMARY KEY,
  nome_produto VARCHAR(100) NOT NULL,
  id_categoria INT NOT NULL,
  CONSTRAINT fk_prod_categoria FOREIGN KEY (id_categoria) REFERENCES categoria (id_categoria)
);

CREATE TABLE venda (
  id_venda   INT PRIMARY KEY,
  id_cliente INT NOT NULL,
  CONSTRAINT fk_venda_cliente FOREIGN KEY (id_cliente) REFERENCES cliente (id_cliente)
);

CREATE TABLE item_venda (
  id_venda   INT NOT NULL,
  id_produto INT NOT NULL,
  qtd        INT NOT NULL,
  PRIMARY KEY (id_venda, id_produto),
  CONSTRAINT fk_iv_venda   FOREIGN KEY (id_venda)   REFERENCES venda   (id_venda),
  CONSTRAINT fk_iv_produto FOREIGN KEY (id_produto) REFERENCES produto (id_produto),
  CONSTRAINT ck_iv_qtd CHECK (qtd > 0)
);

INSERT INTO cliente   VALUES (10,'Ana'),(11,'Rui');
INSERT INTO categoria VALUES (2,'Periféricos'),(3,'Livros');
INSERT INTO produto   VALUES (7,'Mouse',2),(8,'Webcam',2),(9,'Livro SQL',3);
INSERT INTO venda     VALUES (1001,10),(1002,11);
INSERT INTO item_venda VALUES (1001,7,2),(1001,8,1),(1002,9,1);
```

Validação: reconstrói o relatório original (3 linhas):

```sql
-- Validação: reconstrói o relatório original (3 linhas)
SELECT v.id_venda AS venda, c.id_cliente AS cliente, c.nome_cliente,
       p.id_produto AS produto, p.nome_produto,
       ca.id_categoria AS categoria, ca.nome_categoria, i.qtd
FROM item_venda i
JOIN venda v      ON v.id_venda      = i.id_venda
JOIN cliente c    ON c.id_cliente    = v.id_cliente
JOIN produto p    ON p.id_produto    = i.id_produto
JOIN categoria ca ON ca.id_categoria = p.id_categoria
ORDER BY v.id_venda, p.id_produto;
```

Result Grid Filter Rows: Export: Wrap Cell Content: IA

venda	cliente	nome_cliente	produto	nome_produto	categoria	nome_categoria	qtd
1001	10	Ana	7	Mouse	2	Periféricos	2
1001	10	Ana	8	Webcam	2	Periféricos	1
1002	11	Rui	9	Livro SQL	3	Livros	1

Exercício 8:

Orientação acadêmica: 3FN fora da BCNF:

DFs: {aluno, disciplina} -> professor e professor -> disciplina. A relação está em 3FN (disciplina é atributo primo), mas professor não é superchave, então viola a BCNF. Decomposição em professor_disciplina e orientacao.

```
327 • DROP DATABASE IF EXISTS t10_ex08;
328 • CREATE DATABASE t10_ex08;
329 • USE t10_ex08;
330
331   -- professor -> disciplina (professor é chave)
332 • CREATE TABLE professor_disciplina (
333     professor  VARCHAR(60) PRIMARY KEY,
334     disciplina VARCHAR(30) NOT NULL
335   );
336
337   -- aluno x professor (chave composta)
338 • CREATE TABLE orientacao (
339     aluno     INT         NOT NULL,
340     professor VARCHAR(60) NOT NULL,
341     PRIMARY KEY (aluno, professor),
342     CONSTRAINT fk_ori_prof FOREIGN KEY (professor) REFERENCES professor_disciplina (professor)
343   );
344
345 • INSERT INTO professor_disciplina VALUES ('Carla','BD'),('Diego','IA'),('Elisa','Redes');
346 • INSERT INTO orientacao VALUES (1,'Carla'),(1,'Diego'),(2,'Carla'),(3,'Elisa');
```

Validação: reconstrói as orientações por JOIN (4 linhas, sem espúrias):

```
348   -- Validação: reconstrói as orientações por JOIN (4 linhas, sem espúrias)
349 • SELECT o.aluno, pd.disciplina, o.professor
350   FROM orientacao o
351   JOIN professor_disciplina pd ON pd.professor = o.professor
352   ORDER BY o.aluno, pd.disciplina;
```

Result Grid Filter Rows: Export: Wrap Cell Content: IA

aluno	disciplina	professor
1	BD	Carla
1	IA	Diego
2	BD	Carla
3	Redes	Elisa

A DF {aluno, disciplina} -> professor não é preservada pelas constraints (custo conhecido da BCNF). Verificação de violação (deve retornar 0 linhas):

```
354   -- Observação: a DF {aluno, disciplina} -> professor não é preservada
355   -- pelas constraints (custo conhecido da BCNF). Verificação de violação:
356 • SELECT o.aluno, pd.disciplina, COUNT(*) AS qtd_professores
357   FROM orientacao o
358   JOIN professor_disciplina pd ON pd.professor = o.professor
359   GROUP BY o.aluno, pd.disciplina
360   HAVING COUNT(*) > 1;    -- deve retornar 0 linhas
```

Result Grid Filter Rows: Export: Wrap Cell Content: IA

aluno	disciplina	qtd_professores

Exercício 9:

Reserva de laboratório e responsável (BCNF):

DFs: {turma, turno, laboratorio} -> responsavel e {responsavel, turno} -> laboratorio. O determinante {responsavel, turno} não é superchave, violando a BCNF. O laboratório passa para **alocacao_responsavel** e a reserva referencia (responsavel, turno).

```
367 • DROP DATABASE IF EXISTS t10_ex09;
368 • CREATE DATABASE t10_ex09;
369 • USE t10_ex09;
370
371   -- {responsavel, turno} -> laboratorio
372 • CREATE TABLE alocacao_responsavel (
373     responsavel VARCHAR(60) NOT NULL,
374     turno       VARCHAR(10) NOT NULL,
375     laboratorio VARCHAR(10) NOT NULL,
376     PRIMARY KEY (responsavel, turno),
377     CONSTRAINT ck_aloc_turno CHECK (turno IN ('MANHÃ','TARDE','NOITE'))
378   );
379
380 • CREATE TABLE reserva (
381     turma       VARCHAR(10) NOT NULL,
382     turno       VARCHAR(10) NOT NULL,
383     responsavel VARCHAR(60) NOT NULL,
384     PRIMARY KEY (turma, turno, responsavel),
385     CONSTRAINT fk_res_aloc FOREIGN KEY (responsavel, turno)
386       REFERENCES alocacao_responsavel (responsavel, turno)
387   );
388
389 • INSERT INTO alocacao_responsavel VALUES
390     ('Ana','NOITE','L201'),
391     ('Bruno','MANHÃ','L105'),
392     ('Carla','MANHÃ','L110');
393 • INSERT INTO reserva VALUES
394     ('T1','NOITE','Ana'),
395     ('T2','NOITE','Ana'),
396     ('T3','MANHÃ','Bruno'),
397     ('T1','MANHÃ','Carla');
```

Validação: reservas reconstruídas (4 linhas = relação original):

```
399   -- Validação: reservas reconstruídas (4 linhas = relação original)
400 • SELECT r.turma, r.turno, r.responsavel, a.laboratorio
401   FROM reserva r
402   JOIN alocacao_responsavel a
403     ON a.responsavel = r.responsavel AND a.turno = r.turno
404   ORDER BY r.turma, r.turno;
```

Result Grid Filter Rows: Export: Wrap Cell Content: IA

turma	turno	responsavel	laboratorio
T1	MANHÃ	Carla	L110
T1	NOITE	Ana	L201
T2	NOITE	Ana	L201
T3	MANHÃ	Bruno	L105

Exercício 10:

Histórico de pedidos e catálogo (1FN -> BCNF):

Cliente, categoria, fornecedor e produto viraram cadastros próprios; o pedido guarda o cliente e o item guarda qtd e preco_unitario. O preço fica no item porque é o preço praticado naquele pedido (histórico), e não o preço atual do catálogo.

```
409 • DROP DATABASE IF EXISTS t10_ex10;
410 • CREATE DATABASE t10_ex10;
411 • USE t10_ex10;
412
413 • CREATE TABLE cliente (
414     id_cliente INT AUTO_INCREMENT PRIMARY KEY,
415     nome       VARCHAR(100) NOT NULL UNIQUE
416   );
417
418 • CREATE TABLE categoria (
419     id_categoria INT AUTO_INCREMENT PRIMARY KEY,
420     nome         VARCHAR(60) NOT NULL UNIQUE
421   );
422
423 • CREATE TABLE fornecedor (
424     id_fornecedor INT AUTO_INCREMENT PRIMARY KEY,
425     nome          VARCHAR(100) NOT NULL UNIQUE
426   );
427
428 • CREATE TABLE produto (
429     id_produto    INT AUTO_INCREMENT PRIMARY KEY,
430     nome          VARCHAR(100) NOT NULL UNIQUE,
431     id_categoria  INT NOT NULL,
432     id_fornecedor INT NOT NULL,
433     CONSTRAINT fk_prod_cat  FOREIGN KEY (id_categoria)  REFERENCES categoria  (id_categoria),
434     CONSTRAINT fk_prod_forn FOREIGN KEY (id_fornecedor) REFERENCES fornecedor (id_fornecedor)
435   );
436
437 • CREATE TABLE pedido (
438     id_pedido  INT PRIMARY KEY,
439     id_cliente INT NOT NULL,
440     CONSTRAINT fk_ped_cliente FOREIGN KEY (id_cliente) REFERENCES cliente (id_cliente)
441   );
442
443   -- preco_unitario fica no item: é o preço praticado naquele pedido (histórico)
444 • CREATE TABLE item_pedido (
445     id_pedido      INT NOT NULL,
446     id_produto     INT NOT NULL,
447     qtd            INT NOT NULL,
448     preco_unitario DECIMAL(10,2) NOT NULL,
449     PRIMARY KEY (id_pedido, id_produto),
450     CONSTRAINT fk_ip_pedido  FOREIGN KEY (id_pedido)  REFERENCES pedido  (id_pedido),
451     CONSTRAINT fk_ip_produto FOREIGN KEY (id_produto) REFERENCES produto (id_produto),
452     CONSTRAINT ck_ip_qtd   CHECK (qtd > 0),
453     CONSTRAINT ck_ip_preco CHECK (preco_unitario >= 0)
454   );
455
456 • INSERT INTO cliente    (id_cliente, nome) VALUES (1,'Ana'),(2,'Rui');
457 • INSERT INTO categoria  (id_categoria, nome) VALUES (1,'Perif.'),(2,'Livros');
458 • INSERT INTO fornecedor (id_fornecedor, nome) VALUES (1,'Tech A'),(2,'Tech B'),(3,'Editora X');
459 • INSERT INTO produto (id_produto, nome, id_categoria, id_fornecedor) VALUES
460     (1,'Mouse',1,1),(2,'Webcam',1,2),(3,'Livro SQL',2,3);
461 • INSERT INTO pedido VALUES (5001,1),(5002,2);
462 • INSERT INTO item_pedido VALUES
463     (5001,1,2,80.00),
464     (5001,2,1,150.00),
465     (5002,3,1,90.00);
```

Validação 1: reconstrói o histórico (3 linhas):

```sql
-- Validação 1: reconstrói o histórico (3 linhas)
SELECT pe.id_pedido AS pedido, c.nome AS cliente, pr.nome AS produto,
       ca.nome AS categoria, f.nome AS fornecedor,
       i.qtd, i.preco_unitario AS preco, i.qtd * i.preco_unitario AS subtotal
FROM item_pedido i
JOIN pedido pe     ON pe.id_pedido     = i.id_pedido
JOIN cliente c     ON c.id_cliente     = pe.id_cliente
JOIN produto pr    ON pr.id_produto    = i.id_produto
JOIN categoria ca  ON ca.id_categoria  = pr.id_categoria
JOIN fornecedor f  ON f.id_fornecedor  = pr.id_fornecedor
ORDER BY pe.id_pedido, pr.nome;
```

Result Grid Filter Rows: Export: Wrap Cell Content: IA

pedido	cliente	produto	categoria	fornecedor	qtd	preco	subtotal
5001	Ana	Mouse	Perif.	Tech A	2	80.00	160.00
5001	Ana	Webcam	Perif.	Tech B	1	150.00	150.00
5002	Rui	Livro SQL	Livros	Editora X	1	90.00	90.00

Validação 2: total do pedido (5001 -> 310.00 | 5002 -> 90.00):

```sql
-- Validação 2: total do pedido (5001 -> 310.00 | 5002 -> 90.00)
SELECT i.id_pedido, SUM(i.qtd * i.preco_unitario) AS total_pedido
FROM item_pedido i
GROUP BY i.id_pedido
ORDER BY i.id_pedido;
```

Result Grid Filter Rows: Export: Wrap Cell Content: IA

id_pedido	total_pedido
5001	310.00
5002	90.00